

TEST FOR ANYLOGISTIX API

The API info is here: <https://anylogistix.help/api/python/readme.html>

PHASE 1 CONNECTION AND SIMULATION WITH THE ANYLOGISTIX API

Install the dependencies

It assumes that the openapi_client is in the local directory ./openapi-python-client-3.3.1

```
In [5]: import sys
import os
# This is my personal directory
os.chdir(r"C:\Users\Luis\Downloads\TFG\API")
#!{sys.executable} -m pip install "pydantic>=2.10,<3"
!{sys.executable} -m pip install "pydantic<2,>=1.10.5"
!{sys.executable} -m pip install -e ./openapi-python-client-3.3.1
```

```

Requirement already satisfied: pydantic<2,>=1.10.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (1.10.26)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from pydantic<2,>=1.10.5) (4.15.0)
Obtaining file:///C:/Users/Luis/Downloads/TFG/API/openapi-python-client-3.3.1
  Installing build dependencies: started
  Installing build dependencies: finished with status 'done'
  Checking if build backend supports build_editable: started
  Checking if build backend supports build_editable: finished with status 'done'
  Getting requirements to build editable: started
  Getting requirements to build editable: finished with status 'done'
  Preparing editable metadata (pyproject.toml): started
  Preparing editable metadata (pyproject.toml): finished with status 'done'
Requirement already satisfied: urllib3<2.1.0,>=1.25.3 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (2.0.7)
Requirement already satisfied: python-dateutil in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (2.9.0.post0)
Requirement already satisfied: pydantic<2,>=1.10.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (1.10.26)
Requirement already satisfied: aenum in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (3.1.17)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from pydantic<2,>=1.10.5->openapi-client==1.0.0) (4.15.0)
Requirement already satisfied: six>=1.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from python-dateutil->openapi-client==1.0.0) (1.17.0)
Building wheels for collected packages: openapi-client
  Building editable for openapi-client (pyproject.toml): started
  Building editable for openapi-client (pyproject.toml): finished with status 'done'
  Created wheel for openapi-client: filename=openapi_client-1.0.0-0.editable-py3-none-any.whl size=3042 sha256=6149f00765df5e532a9cb8866eca36fcadfd6ab22763beb7583650ea00d87f23
  Stored in directory: C:\Users\Luis\AppData\Local\Temp\pip-ephem-wheel-cache-3jfab27u\wheels\96\67\f9\218cef519d1de93e10ecbd2f12fb5f4d1d0e81b1c1d5296ba2
Successfully built openapi-client
Installing collected packages: openapi-client
  Attempting uninstall: openapi-client
    Found existing installation: openapi-client 1.0.0
    Uninstalling openapi-client-1.0.0:
      Successfully uninstalled openapi-client-1.0.0
Successfully installed openapi-client-1.0.0

```

```

In [6]: import openapi_client
        from openapi_client.rest import ApiException
        from pprint import pprint
        import urllib3

```

```

In [7]: # SERVER_IP="192.168.64.159"
        # SERVER_IP="192.168.67.110"
        SERVER_IP="alxserver.aut.uah.es"
        SERVER_URL=f"https://{SERVER_IP}:443/api/v1"

        # This is my personal APY KEY
        API_KEY="c184f1ab-9f13-484c-a1c1-3d543502da6e"

```

```
SERVER_URL
```

```
Out[7]: 'https://alxserver.aut.uah.es:443/api/v1'
```

PART I: CHECK CONNECTIVITY AND OPERATIONAL STATUS

```
In [8]: # Defining the host is optional and defaults to https://server_address:port/api/
# See configuration.py for a list of all supported configuration parameters.
configuration = openapi_client.Configuration(
    host = SERVER_URL
)
# The client must configure the authentication and authorization parameters
# in accordance with the API server security policy.
# Examples for each auth method are provided below, use the example that
# satisfies your auth use case.
# Configure API key authorization: ApiKey
configuration.api_key['ApiKey'] = API_KEY

# ALVARO: SUPER IMPORTANTE PARA PODER TRABAJAR CON CERTIFICADOS AUTO-FIRMADOS
configuration.verify_ssl = False
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
with openapi_client.ApiClient(configuration) as api_client:
    # Create an instance of the API class
    api_instance = openapi_client.OpenApiApi(api_client)
```

Obtain current user data

```
In [9]: try:
    # Gets current user data.
    api_response = api_instance.get_current_user()
    print("The response of OpenApiApi->get_current_user:\n")
    pprint(api_response)
except Exception as e:
    print("Exception when calling OpenApiApi->get_current_user: %s\n" % e)
```

The response of OpenApiApi->get_current_user:

```
ApiUserData(email='adrian.encabo@edu.uah.es', first_name='adrian.encabo@edu.uah.e
s', id=403, last_name='', username='adrian.encabo@edu.uah.es')
```

PART II: OBTAIN PROJECTS, SCENARIOS, EXECUTIONS

Obtain the list of currently available projects

```
In [10]: try:
    # Returns a list of projects that the user has access to.
    api_response = api_instance.get_projects()
    print("The response of OpenApiApi->get_projects:\n")
    pprint(api_response)
except Exception as e:
    print("Exception when calling OpenApiApi->get_projects: %s\n" % e)
```

The response of OpenApiApi->get_projects:

```
[ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=403, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_use
r_id=3)]
```

Open a project and get the list of scenarios available

```
In [11]: # My personal Project defined
MY_PROJECT_NAME="TFG_ADRIAN_ENCABO"
the_project_id=0
try:
    full_match = True # bool / Whether to match project name by complete match.
    # Opens project with name projectName.
    the_project = api_instance.find_and_open_project_by_name(full_match, MY_PROJ
    print("The response of OpenApiApi->find_and_open_project_by_name:\n")
    pprint(the_project)
except Exception as e:
    print("Exception when calling OpenApiApi->find_and_open_project_by_name: %s\
try:
    # Returns a List of scenarios for the project with identifier project_id.
    the_scenarios = api_instance.get_scenarios(the_project.id)
    print("The response of OpenApiApi->get_scenarios:\n")
    pprint(the_scenarios)
except Exception as e:
    print("Exception when calling OpenApiApi->get_scenarios: %s\n" % e)
```

The response of OpenApiApi->find_and_open_project_by_name:

```
ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=403, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)
```

The response of OpenApiApi->get_scenarios:

```
[ApiScenarioData(id=82840, name='Budget Comparison (Estimated Demand)', type='SIM'),
 ApiScenarioData(id=87083, name='P3.3. GFA 1. Results 2_Different Locations', type='SIM'),
 ApiScenarioData(id=84191, name='Budget Comparison (Estimated Demand) 2', type='SIM'),
 ApiScenarioData(id=85538, name='Budget Comparison (Estimated Demand) 2', type='SIM'),
 ApiScenarioData(id=86671, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction', type='NO'),
 ApiScenarioData(id=88185, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction 2', type='NO'),
 ApiScenarioData(id=89212, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction sim', type='SIM'),
 ApiScenarioData(id=92735, name='P4.2.6 SIM Distribution Network-4 Walls_New Inventory+New Transportation', type='SIM'),
 ApiScenarioData(id=95044, name='null', type='SIM'),
 ApiScenarioData(id=176431, name='Cold Chain (Risk Analysis)', type='SIM'),
 ApiScenarioData(id=177767, name='US Distribution Network', type='NO'),
 ApiScenarioData(id=178297, name='Lumber Production', type='NO'),
 ApiScenarioData(id=179567, name='Global Network Examination', type='SIM'),
 ApiScenarioData(id=180669, name='asset-v1_UAH+EUGLOH+EUG2_T2_1_0080+type@asset+block@P4.4_Demand_Variation_LoadingGates_2', type='SIM'),
 ApiScenarioData(id=181707, name='P3.1._Green_Field_Analysis_GFA__ALX3', type='GFA'),
 ApiScenarioData(id=192031, name='null 6', type='SIM'),
 ApiScenarioData(id=182396, name='null 2', type='SIM'),
 ApiScenarioData(id=187186, name='null 4', type='SIM'),
 ApiScenarioData(id=184791, name='null 3', type='SIM'),
 ApiScenarioData(id=189602, name='null 5', type='SIM'),
 ApiScenarioData(id=194446, name='null 7', type='SIM')]
```

Select the scenario by name

```
In [27]: #Add the name of the scenario
MY_SCENARIO_NAME="Global Network Examination"

# 1. Select the scenario
the_scenario = next(s for s in the_scenarios if s.name == MY_SCENARIO_NAME)
pprint(the_scenario)
```

```
ApiScenarioData(id=179567, name='Global Network Examination', type='SIM')
```

Get the list of experiment runs of a scenario

```
In [28]: try:
# Returns the results of experiment runs for scenario.
the_runs = api_instance.get_experiment_runs_for_scenario(the_scenario.id)
print("The response of OpenApiApi->get_experiment_runs_for_scenario:\n")
pprint(the_runs)
```

```
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiment_runs_for_scenario:
```

The response of OpenApiApi->get_experiment_runs_for_scenario:

```
[ApiBasicRun(has_options=False, id=201331, name='Statistics', rc_id=201332, removing=False, scenario_id=179567, type='SIMULATION')]
```

PART III. EXECUTE A SIMULATION

Get the list of RunConfigurations for the experiments of a given Scenario

Each type of experiment has a RunConfiguration. For example:

```
ApiBasicRunConfiguration(animation_model_id=None,
feasible_check_status='NOT_CONDUCTED', id=2439,
is_feasible_check_data_available=False, model_time=None,
name='i18n-Experiment_Simulation_text', progress=None,
project_id=63, routes_progress=None, run_status=None,
scenario_id=1752, speed=None, status='IDLE', step='IDLE',
type='SIMULATION', validation_status='VALID')
```

Types of available experiment runs are:

- SIMULATION
- VARIATION
- COMPARISON
- SAFETY_STOCK
- RISK_ANALYSIS

```
In [29]: MY_RUN_TYPE="SIMULATION"
try:
    # Returns a list of experiments available for a given scenario.
    the_run_configurations= api_instance.get_experiments(the_project.id, the_scenario_id)
    print("The response of OpenApiApi->get_experiments:\n")
    # Exception fixed for undefined variable
    pprint(the_run_configurations)
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiments: %s\n" % e)

# 2. Select the adequate RunConfiguration for the scenario
the_RC = next((r for r in the_run_configurations if r.type == MY_RUN_TYPE), None)
pprint(the_RC)
```

The response of OpenApiApi->get_experiments:

```
[ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=180254, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_Simulation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='SIMULATION', validation_status='VALID'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=180429, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_VariationExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='VARIATION', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=179575, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_ComparisonExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='COMPARISON', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=180115, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_SafetyStockEstimation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='SAFETY_STOCK', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=179930, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_RiskAnalysisExperiment_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='RISK_ANALYSIS', validation_status='NOT_CONDUCTED')]
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CONDUCTED', id=180254, is_feasible_check_data_available=False, model_time=None, name='i18n-Experiment_Simulation_text', progress=None, project_id=79020, routes_progress=None, run_status=None, scenario_id=179567, speed=None, status='IDLE', step='IDLE', type='SIMULATION', validation_status='VALID')
```

Run a new experiment over a scenario (synch)

Runs synchronously.

```
In [30]: try:
          # Starts the experiment synchronously.
          the_sync_result = api_instance.run_experiment_synchronously(the_RC.id)
          print("The response of OpenApiApi->run_experiment_synchronously:\n")
          pprint(the_sync_result)
        except Exception as e:
          print("Exception when calling OpenApiApi->run_experiment_synchronously: %s\n"
```

The response of OpenApiApi->run_experiment_synchronously:

```
ApiExperimentResult(experiment_result_id=201579, validation_errors=None, validation_status='OK', validation_warnings=None)
```

Part IIIb. Async execution

Run a new experiment over a scenario (asynch)

Runs asynchronously. Periodic status check.

```
In [31]: skip_scenario_warnings = True # bool | Whether to skip scenario warnings. (optional)
try:
    # Starts the experiment asynchronously.
    the_execution = api_instance.run_experiment(the_RC.id, skip_scenario_warning
    print("The response of OpenApiApi->run_experiment:\n")
    pprint(the_execution)
except Exception as e:
    print("Exception when calling OpenApiApi->run_experiment: %s\n" % e)

import time
while True:
    try:
        # Returns the experiment status
        the_status = api_instance.get_experiment_status(the_RC.id)
        # pprint(api_response)

        # Check if execution finished
        if the_status.finished:
            print("Execution finished.")
            break

        # Optional: stop if failed
        # fixed exception of status check
        if the_status.failed:
            print("Execution failed.")
            break

    except Exception as e:
        print("Exception when calling OpenApiApi->get_experiment_status:", e)
    print("Not finished yet. Wait 1sec and check again")
    # wait before next check
    time.sleep(1)
```

The response of OpenApiApi->run_experiment:

```
ApiRunExperiment(animation_model_id=180254, experiment_type_id='SIMULATION', is3d
_animation=False)
Execution finished.
```

Get the experiments results (pages)

You can ask for limited number of results (avoid overflow)

```
In [32]: skip = 0 # int | Number of records to skip. (default to 0)
take = 1000 # int | Number of records to be retrieved. (default to 1000)

try:
    # Returns all the results of the specific experiment run.
    the_run_results = api_instance.get_experiment_run_result(the_RC.id, skip, ta
    print("The response of OpenApiApi->get_experiment_run_result:\n")
    pprint(the_run_results)
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiment_run_result: %s\n" %
```

The response of OpenApiApi->get_experiment_run_result:

```
ApiExperimentResultData(pages=[ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180257, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Available Inventory', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180257, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180258, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=1, width=2), name='Available Inventory Including Backlog', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180258, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180259, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Average Daily Available Inventory', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180259, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180260, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='Average Daily On-hand Inventory', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180260, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180261, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='On-hand Inventory', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180261, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='180256', name='Available Inventory')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180263, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Demand Placed, Fulfillment Received (Accumulative)', rc_id=180254, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=180263, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180264, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=1, width=2), name='Demand Placed, Fulfillment Received (Accumulative, Per Object)', rc_id=180254, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=180264, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180265, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Demand Placed, Fulfillment Received (Daily)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180265, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180266, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='Demand Placed, Fulfillment Received (Daily, Per Object)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180266, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180267, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Demand Received, Fulfillment Shipped (Accumulative)', rc_id=180254, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=180267, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180268, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=1, width=2), name='Demand Received, Fulfillment
```

```

ent Shipped (Daily)', rc_id=180254, type='LINE')), class_name='ApiExperimentResult
GraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180268, show_x=Tru
e, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPag
e(id='180262', name='Fulfillment')), ApiExperimentResultPageWrapper(charts=[ApiEx
perimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180270, is_group_m
ode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0,
width=2), name='Lead Time', rc_id=180254, type='LINE')), class_name='ApiExperim
entResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180270, show_
x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExpe
rimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180271, is_group_mod
e_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, wi
dth=2), name='Max Lead Time', rc_id=180254, type='TABLE')), class_name='ApiExperim
entResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=1802
71, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperim
entResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180272, is_group_mode_e
nabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width
=2), name='Mean Lead Time (Best-Mean-Worst)', rc_id=180254, type='BMW-LINE')), cla
ss_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_lis
t=[], id=180272, show_x=True, total_entity_count=0, total_technical_row_count=0,
type='BMW-LINE'))], page=ApiPage(id='180269', name='Lead Time')), ApiExperimentRe
sultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetada
taShort(id=180274, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=
1, start_col=4, start_row=0, width=2), name='Available Inventory Including Backlo
g', rc_id=180254, type='LINE')), class_name='ApiExperimentResultGraphChartWraphe
r', data=ApiGraphChartData(entity_list=[], id=180274, show_x=True, total_entity_c
ount=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChart
Wrapper(chart=ApiChartMetadataShort(id=180277, is_group_mode_enabled=False, layou
t=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Products
Produced (Units) Factory Shenzhen', rc_id=180254, type='BAR')), class_name='ApiExp
erimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=18027
7, show_x=True, total_entity_count=0, total_technical_row_count=0, type='BAR')),
ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180280, is_gr
oup_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_ro
w=0, width=2), name='Products Produced (Units) Factory Xiamen', rc_id=180254, typ
e='BAR')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartDa
ta(entity_list=[], id=180280, show_x=True, total_entity_count=0, total_technical_
row_count=0, type='BAR'))], page=ApiPage(id='180273', name='Production')), ApiExp
erimentResultPageWrapper(charts=[ApiExperimentResultTableChartWrapper(chart=ApiCh
artMetadataShort(id=180284, is_group_mode_enabled=False, layout=ApiChartLayoutDat
a(height=1, start_col=2, start_row=0, width=2), name='Profit and Loss Statement',
rc_id=180254, type='TABLE')), class_name='ApiExperimentResultTableChartWrapper', d
ata=ApiStrictTableChartData(entity_list=[], id=180284, total_entity_count=0, tota
l_technical_row_count=0, type='TABLE')), ApiExperimentResultGraphChartWrapper(ch
art=ApiChartMetadataShort(id=180285, is_group_mode_enabled=False, layout=ApiChartL
ayoutData(height=1, start_col=4, start_row=0, width=2), name='Profit, Revenue, To
tal Cost', rc_id=180254, type='BAR')), class_name='ApiExperimentResultGraphChartWr
apper', data=ApiGraphChartData(entity_list=[], id=180285, show_x=True, total_enti
ty_count=0, total_technical_row_count=0, type='BAR')), ApiExperimentResultGraphCh
artWrapper(chart=ApiChartMetadataShort(id=180286, is_group_mode_enabled=False, la
yout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Reven
ue, Total Cost', rc_id=180254, type='LINE')), class_name='ApiExperimentResultGraph
ChartWrapper', data=ApiGraphChartData(entity_list=[], id=180286, show_x=True, tot
al_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id
='180283', name='Profit and Loss Statement')), ApiExperimentResultPageWrapper(ch
arts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180288,
is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, sta
rt_row=1, width=2), name='ELT Service Level by Products (Per Product)', rc_id=180
254, type='LINE')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGra
phChartData(entity_list=[], id=180288, show_x=True, total_entity_count=0, total_t
echnical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=A

```

```

piChartMetadataShort(id=180289, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='ELT Service Level by Products (Per Source)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180289, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180290, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=1, width=2), name='ELT Service Level by Revenue (Per Object)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180290, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180291, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Service Level by Products (Per Product)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180291, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180292, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Service Level by Products (Per Source)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180292, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180293, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Service Level by Revenue (Per Object)', rc_id=180254, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180293, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE'))], page=ApiPage(id='180287', name='Service Level')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=180295, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=3, start_row=0, width=3), name='Customer Tariffs', rc_id=180254, type='BAR'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=180295, show_x=True, total_entity_count=0, total_technical_row_count=0, type='BAR')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=180296, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=3), name='Customer Tariffs, Profit, Total Cost, Revenue', rc_id=180254, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=180296, total_entity_count=0, total_technical_row_count=0, type='TABLE'))], page=ApiPage(id='180294', name='Tariffs'))))

```

```

In [33]: for p in the_run_results.pages:
          print(f"\n{p.page.name} (id:{p.page.id})")
          for c in p.charts:
              print(f"    - {c.chart.name} (type: {c.chart.type}) (id: {c.chart.id})")

```

Available Inventory (id:180256)

- Available Inventory (type: LINE) (id: 180257)
- Available Inventory Including Backlog (type: LINE) (id: 180258)
- Average Daily Available Inventory (type: LINE) (id: 180259)
- Average Daily On-hand Inventory (type: LINE) (id: 180260)
- On-hand Inventory (type: LINE) (id: 180261)

Fulfillment (id:180262)

- Demand Placed, Fulfillment Received (Accumulative) (type: TABLE) (id: 180263)
- Demand Placed, Fulfillment Received (Accumulative, Per Object) (type: TABLE) (id: 180264)
- Demand Placed, Fulfillment Received (Daily) (type: LINE) (id: 180265)
- Demand Placed, Fulfillment Received (Daily, Per Object) (type: LINE) (id: 180266)
- Demand Received, Fulfillment Shipped (Accumulative) (type: TABLE) (id: 180267)
- Demand Received, Fulfillment Shipped (Daily) (type: LINE) (id: 180268)

Lead Time (id:180269)

- Lead Time (type: LINE) (id: 180270)
- Max Lead Time (type: TABLE) (id: 180271)
- Mean Lead Time (Best-Mean-Worst) (type: BMW-LINE) (id: 180272)

Production (id:180273)

- Available Inventory Including Backlog (type: LINE) (id: 180274)
- Products Produced (Units) Factory Shenzhen (type: BAR) (id: 180277)
- Products Produced (Units) Factory Xiamen (type: BAR) (id: 180280)

Profit and Loss Statement (id:180283)

- Profit and Loss Statement (type: TABLE) (id: 180284)
- Profit, Revenue, Total Cost (type: BAR) (id: 180285)
- Revenue, Total Cost (type: LINE) (id: 180286)

Service Level (id:180287)

- ELT Service Level by Products (Per Product) (type: LINE) (id: 180288)
- ELT Service Level by Products (Per Source) (type: LINE) (id: 180289)
- ELT Service Level by Revenue (Per Object) (type: LINE) (id: 180290)
- Service Level by Products (Per Product) (type: LINE) (id: 180291)
- Service Level by Products (Per Source) (type: LINE) (id: 180292)
- Service Level by Revenue (Per Object) (type: LINE) (id: 180293)

Tariffs (id:180294)

- Customer Tariffs (type: BAR) (id: 180295)
- Customer Tariffs, Profit, Total Cost, Revenue (type: TABLE) (id: 180296)

Obtain the pages in the experiment's result dashboard

```
In [34]: try:
# Returns statistics pages for the result of experiment run.
# Exception fixed for 'id' attribute
# We use the experiment_result_id from the previously executed synchronous r
result_id = the_sync_result.experiment_result_id
dashboard_pages = api_instance.get_experiment_dashboard_pages(result_id)
print("The response of OpenApiApi->get_experiment_dashboard_pages:\n")

# CHANGED Displaying the available pages and charts with their dynamic IDs
for page in dashboard_pages:
    print(f"Dashboard Page: {page.name} (ID: {page.id})")
    for chart in page.charts:
```

```
print(f" - Chart: {chart.name} (ID: {chart.id})")

except Exception as e:
    print("Exception when calling OpenApiApi->get_experiment_dashboard_pages: %s"
```

The response of OpenApiApi->get_experiment_dashboard_pages:

Dashboard Page: Available Inventory (ID: 201581)

- Chart: Available Inventory (ID: 201582)
- Chart: Available Inventory Including Backlog (ID: 201583)
- Chart: Average Daily Available Inventory (ID: 201584)
- Chart: Average Daily On-hand Inventory (ID: 201585)
- Chart: On-hand Inventory (ID: 201586)

Dashboard Page: Fulfillment (ID: 201587)

- Chart: Demand Placed, Fulfillment Received (Accumulative) (ID: 201588)
- Chart: Demand Placed, Fulfillment Received (Accumulative, Per Object) (ID: 201589)
- Chart: Demand Placed, Fulfillment Received (Daily) (ID: 201590)
- Chart: Demand Placed, Fulfillment Received (Daily, Per Object) (ID: 201591)
- Chart: Demand Received, Fulfillment Shipped (Accumulative) (ID: 201592)
- Chart: Demand Received, Fulfillment Shipped (Daily) (ID: 201593)

Dashboard Page: Lead Time (ID: 201594)

- Chart: Lead Time (ID: 201595)
- Chart: Max Lead Time (ID: 201596)
- Chart: Mean Lead Time (Best-Mean-Worst) (ID: 201597)

Dashboard Page: Production (ID: 201598)

- Chart: Available Inventory Including Backlog (ID: 201599)
- Chart: Products Produced (Units) Factory Shenzhen (ID: 201602)
- Chart: Products Produced (Units) Factory Xiamen (ID: 201605)

Dashboard Page: Profit and Loss Statement (ID: 201608)

- Chart: Profit and Loss Statement (ID: 201609)
- Chart: Profit, Revenue, Total Cost (ID: 201610)
- Chart: Revenue, Total Cost (ID: 201611)

Dashboard Page: Service Level (ID: 201612)

- Chart: ELT Service Level by Products (Per Product) (ID: 201613)
- Chart: ELT Service Level by Products (Per Source) (ID: 201614)
- Chart: ELT Service Level by Revenue (Per Object) (ID: 201615)
- Chart: Service Level by Products (Per Product) (ID: 201616)
- Chart: Service Level by Products (Per Source) (ID: 201617)
- Chart: Service Level by Revenue (Per Object) (ID: 201618)

Dashboard Page: Tariffs (ID: 201619)

- Chart: Customer Tariffs (ID: 201620)
- Chart: Customer Tariffs, Profit, Total Cost, Revenue (ID: 201621)

Export the results as an excel matrix

```
In [35]: # Exception fixed for wrong IDs
import shutil #For manipulating excel files
try:
    # Dynamically select the first dashboard page to export to avoid hardcoded ID
    if dashboard_pages and len(dashboard_pages) > 0:
        target_page_id = dashboard_pages[0].id
        target_result_id = the_sync_result.experiment_result_id

    print(f"Exporting dashboard page ID {target_page_id} from result ID {target_result_id}")

    # Returns an Excel representation of the dashboard page
    excel_export = api_instance.export_dashboard_page(target_result_id, target_page_id)
```

```

# Save the exported Excel file locally
output_filename = "Simulation_Results.xlsx"

# Check if the API returns a temporary file path
if isinstance(excel_export, str) and os.path.exists(excel_export):
    shutil.move(excel_export, output_filename)
    print(f"Excel file saved successfully to: {os.path.abspath(output_filename)}")

# Check if the API returns binary data directly
elif isinstance(excel_export, bytes):
    with open(output_filename, "wb") as file:
        file.write(excel_export)
    print(f"Excel file saved successfully to: {os.path.abspath(output_filename)}")

else:
    print(f"Unexpected data format received. Type: {type(excel_export)}")
    pprint(excel_export)

print("\nData exported successfully.")
else:
    print("No dashboard pages available to export.")

except Exception as e:
    print("Exception when calling OpenApiApi->export_dashboard_page: %s\n" % e)

```

Exporting dashboard page ID 201581 from result ID 201579...

Excel file saved successfully to: C:\Users\Luis\Downloads\TFG\API\Simulation_Results.xlsx

Data exported successfully.

Close current project

In [36]: *#Error fixed of indentation*

```

try:
    # Closes the currently open project.
    api_response = api_instance.close_project()
    print("The response of OpenApiApi->close_project:\n")
    pprint(api_response)
    print("\nProject closed successfully.")
except ApiException as e:
    print("Exception when calling OpenApiApi->close_project: %s\n" % e)

```

The response of OpenApiApi->close_project:

```

ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=None, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)

```

Project closed successfully.

=====

PHASE 2



PART I Flat AI Decision & Scenario Cloning

1. Define the Flat AI logic (3 predefined decisions)

```
In [37]: decision_options = [
    "DECISION 0: Increase Demand by 20% (Aggressive Growth)",
    "DECISION 1: Decrease Transport Costs by 15% (Cost Optimization)",
    "DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)"
]

print("--- FLAT AI: EVALUATING PREDEFINED DECISIONS ---")
for i, decision in enumerate(decision_options):
    print(f"[{i}] {decision}")

# For this 'flat AI' phase, we will simulate the AI choosing a decision automatically
# Let us assume the AI evaluates the current context and selects Decision 0.
selected_decision_index = 0
selected_decision_text = decision_options[selected_decision_index]

print(f"\nFLAT AI SELECTED: {selected_decision_text}")

--- FLAT AI: EVALUATING PREDEFINED DECISIONS ---
[0] DECISION 0: Increase Demand by 20% (Aggressive Growth)
[1] DECISION 1: Decrease Transport Costs by 15% (Cost Optimization)
[2] DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)

FLAT AI SELECTED: DECISION 0: Increase Demand by 20% (Aggressive Growth)
```

2. Clone the existing scenario

```
In [38]: # to apply the decision without altering the base scenario
try:
    print("\nCloning the base scenario to apply the decision safely...")

    # The API requires a body parameter for this request, even if empty
    copy_parameters = openapi_client.ApiConvertAndCopyParameters()

    # We copy the scenario synchronously using the same type as the original.
    cloned_scenario = api_instance.copy_scenario_synchronously(
        new_type=the_scenario.type,
        scenario_id=the_scenario.id,
        body=copy_parameters
    )

    print("The response of OpenApiApi->copy_scenario_synchronously:\n")
    pprint(cloned_scenario)

    print("\nScenario successfully cloned.")
    print(f"New Cloned Scenario ID: {cloned_scenario.id}")
    print(f"New Cloned Scenario Name: {cloned_scenario.name}")

    # We store the new scenario in a specific variable to work with it in the next step
    the_cloned_scenario = cloned_scenario

except Exception as e:
    print("Exception when calling OpenApiApi->copy_scenario_synchronously: %s\n"
```

Cloning the base scenario to apply the decision safely...
The response of OpenApiApi->copy_scenario_synchronously:

```
ApiScenarioData(id=202944, name=None, type='SIM')
```

Scenario successfully cloned.
New Cloned Scenario ID: 202944
New Cloned Scenario Name: None

3. Configure the cloned scenario

```
In [39]: try:
    print(f"--- PREPARING CLONED SCENARIO (ID: {the_cloned_scenario.id}) ---")

    # 1. Retrieve the available experiments for the newly cloned scenario
    print("Fetching available run configurations for the cloned scenario...")
    cloned_run_configurations = api_instance.get_experiments(the_project.id, the

    if cloned_run_configurations:
        # 2. Automatically select the 'SIMULATION' configuration (or the first a
        cloned_RC = next((r for r in cloned_run_configurations if r.type == 'SIM

        print("\nSelected Run Configuration for the cloned scenario successfully
        print(f" - Configuration ID: {cloned_RC.id}")
        print(f" - Configuration Type: {cloned_RC.type}")

        print("\n[SYSTEM] The cloned scenario is now ready to receive data modif

    else:
        print("\nNo run configurations found for the cloned scenario.")
        cloned_RC = None

    except Exception as e:
        print("Exception when configuring the cloned scenario: %s\n" % e)
```

```
--- PREPARING CLONED SCENARIO (ID: 202944) ---
```

```
Fetching available run configurations for the cloned scenario...
```

```
Selected Run Configuration for the cloned scenario successfully:
```

```
- Configuration ID: 203631
- Configuration Type: SIMULATION
```

```
[SYSTEM] The cloned scenario is now ready to receive data modifications.
```

4.a) Modification by Excel

```
In [40]: # This path requires a local excel file. A standard AI agent would use pandas
    # to open it, apply the 20 percent demand increase, and save it.

    # The AI modifies the scenario via Excel upload to retain full Simulation Dashbo

    import os
    import time

    # Replace this with the actual path to your modified Excel file
    modified_excel_path = r"C:\Users\Luis\Downloads\TFG\Curso_logistica_2025\Materia

    print("--- PATH A (STEP 1): EXCEL IMPORT METHOD ---")
```

```

try:
    if os.path.exists(modified_excel_path):
        print(f"Uploading modified Excel file: {modified_excel_path}")

        # FIXED 1: We provide a valid string name, because cloned scenarios have
        # FIXED 2: We pass the file path directly to the 'file' parameter. The A
        api_instance.import_excel_existing(
            new_scenario_name="AI_Decision_0_Scenario",
            project_id=the_project.id,
            scenario_id=the_cloned_scenario.id,
            file=modified_excel_path,
            overwrite_existing_tables=True
        )

        print("\n[SYSTEM] Scenario data updated successfully via Excel import.")

        # We add a small delay to ensure the server database fully registers the
        time.sleep(3)

    else:
        print(f"File '{modified_excel_path}' not found. Please check the path.")

except Exception as e:
    print("Exception when calling OpenApiApi->import_excel_existing: %s\n" % e)

```

--- PATH A (STEP 1): EXCEL IMPORT METHOD ---

Uploading modified Excel file: C:\Users\Luis\Downloads\TFG\Curso_logistica_2025\Material\Tema 4 - MULTIMODAL TRANSPORT PLANNING\2. Practice\Ch2. Simulation Files\P4.2.6 SIM Distribution Network-4 Walls_New Inventory+New Transportation.xlsx

[SYSTEM] Scenario data updated successfully via Excel import.

4.a.2 Run Simulation and export results

```

In [41]: # Now that the scenario is updated via Excel, we run a standard simulation to ge

import os
import shutil
from pprint import pprint

print(f"--- PATH A (STEP 2): RUNNING SIMULATION FOR SCENARIO {the_cloned_scenario}")

try:
    print("1. Fetching SIMULATION run configuration for the updated scenario...")
    # Get experiments for the cloned scenario
    cloned_run_configurations = api_instance.get_experiments(the_project.id, the

    # Select the 'SIMULATION' type
    simulation_RC = next((r for r in cloned_run_configurations if r.type == 'SIM

    if simulation_RC:
        print(f"Found SIMULATION experiment (ID: {simulation_RC.id}). Launching

        # 2. Run the simulation
        sync_result = api_instance.run_experiment_synchronously(simulation_RC.id

        print("\nExperiment finished! Response from server:")
        pprint(sync_result)

```

```

# Safely extract the result ID
result_id = getattr(sync_result, 'experiment_result_id', None)

if result_id:
    print(f"\n[SYSTEM] Simulation finished! Captured Result ID: {result_id}")

    # 3. Fetch the full, rich dashboard pages
    pages = api_instance.get_experiment_dashboard_pages(result_id)

    if pages and len(pages) > 0:
        # Usually the first page has the main KPIs (Financial and Custom)
        target_page = pages[0]
        print(f"\n4. Exporting the rich dashboard page '{target_page.name}'")

        excel_data = api_instance.export_dashboard_page(result_id, target_page)
        output_filename = f"AI_Decision0_Results_Run_{result_id}.xlsx"
        output_path = os.path.join(os.getcwd(), output_filename)

        # 4. Save the final report
        if isinstance(excel_data, str) and os.path.exists(excel_data):
            shutil.move(excel_data, output_path)
            print(f"\n[SUCCESS] Full Simulation Results exported to: {output_path}")
        elif isinstance(excel_data, bytes):
            with open(output_path, "wb") as file:
                file.write(excel_data)
            print(f"\n[SUCCESS] Full Simulation Results exported to: {output_path}")
        else:
            print(f"Unexpected data format received. Type: {type(excel_data)}")
    else:
        print("No dashboard pages found for this simulation result.")
    else:
        print("Failed to capture 'experiment_result_id' from the synchronous result.")
    else:
        print("No SIMULATION experiment found for this scenario.")

except Exception as e:
    print("Exception during Simulation Execution and Export: %s\n" % e)

```

--- PATH A (STEP 2): RUNNING SIMULATION FOR SCENARIO 202944 ---

1. Fetching SIMULATION run configuration for the updated scenario...

Found SIMULATION experiment (ID: 203631). Launching synchronously...

Experiment finished! Response from server:

```
ApiExperimentResult(experiment_result_id=203631, validation_errors=[ApiValidation
Error(message='extension type not found', source=['DC Prague']), ApiValidationErr
or(message='extension type not found', source=['DC Berlin']), ApiValidationError
(message='Site is not defined', source=['Production policy for facility: no facil
ity, for product: no product, for time period: (All periods)']), ApiValidationErr
or(message='Product is not defined', source=['Production policy for facility: no
facility, for product: no product, for time period: (All periods)']), ApiValidati
onError(message='Site is not defined', source=['Production policy for facility: n
o facility, for product: no product, for time period: (All periods)']), ApiValida
tionError(message='Product is not defined', source=['Production policy for facili
ty: no facility, for product: no product, for time period: (All periods)']), ApiV
alidationError(message='Site is not defined', source=['Production policy for faci
lity: no facility, for product: no product, for time period: (All periods)']), Ap
iValidationError(message='Product is not defined', source=['Production policy for
facility: no facility, for product: no product, for time period: (All period
s)']), ApiValidationError(message='Site is not defined', source=['Production poli
cy for facility: no facility, for product: no product, for time period: (All peri
ods)']), ApiValidationError(message='Product is not defined', source=['Production
policy for facility: no facility, for product: no product, for time period: (All
periods)']), ApiValidationError(message='Site is not defined', source=['Productio
n policy for facility: no facility, for product: no product, for time period: (Al
l periods)']), ApiValidationError(message='Product is not defined', source=['Prod
uction policy for facility: no facility, for product: no product, for time perio
d: (All periods)']), ApiValidationError(message='End Product is not defined', sou
rce=['BOM Large home appliances']), ApiValidationError(message='End Product is no
t defined', source=['BOM Lighting']), ApiValidationError(message='End Product is
not defined', source=['BOM Small appliances']), ApiValidationError(message='End P
roduct is not defined', source=['BOM Gardening equipment']), ApiValidationError(m
essage='End Product is not defined', source=['BOM Furniture']), ApiValidationErro
r(message='Site is not defined', source=['Site state change for facility: no faci
lity, for time period: no time period']), ApiValidationError(message='Time Period
is not defined', source=['Site state change for facility: no facility, for time p
eriod: no time period']), ApiValidationError(message='Site is not defined', sourc
e=['Site state change for facility: no facility, for time period: no time perio
d']), ApiValidationError(message='Time Period is not defined', source=['Site stat
e change for facility: no facility, for time period: no time period']), ApiValida
tionError(message='Site is not defined', source=['Site state change for facility:
no facility, for time period: no time period']), ApiValidationError(message='Time
Period is not defined', source=['Site state change for facility: no facility, for
time period: no time period']), ApiValidationError(message='Site is not defined',
source=['Site state change for facility: no facility, for time period: no time pe
riod']), ApiValidationError(message='Time Period is not defined', source=['Site s
tate change for facility: no facility, for time period: no time period']), ApiVal
idationError(message='Site is not defined', source=['Site state change for facili
ty: no facility, for time period: no time period']), ApiValidationError(message
='Time Period is not defined', source=['Site state change for facility: no facili
ty, for time period: no time period']), ApiValidationError(message='Site is not d
efined', source=['Site state change for facility: no facility, for time period: n
o time period']), ApiValidationError(message='Time Period is not defined', source
=['Site state change for facility: no facility, for time period: no time perio
d']), ApiValidationError(message='Site is not defined', source=['Site state chang
e for facility: no facility, for time period: no time period']), ApiValidationErr
or(message='Time Period is not defined', source=['Site state change for facility:
no facility, for time period: no time period']), ApiValidationError(message='Site
is not defined', source=['Site state change for facility: no facility, for time p
```

20/29

[illegible]

[illegible]

[illegible]

[illegible]

25/29

[illegible]

```
d', source=['Tariff: [null - null], for product: no product, for type: Compound
d']], ApiValidationError(message='From is not defined', source=['Tariff: [null -
null], for product: no product, for type: Compound']), ApiValidationError(message
='To is not defined', source=['Tariff: [null - null], for product: no product, fo
r type: Compound']), ApiValidationError(message='Product is not defined', source=
['Tariff: [null - null], for product: no product, for type: Compound']), ApiValid
ationError(message='From is not defined', source=['Tariff: [null - null], for pro
duct: no product, for type: Compound']), ApiValidationError(message='To is not de
fined', source=['Tariff: [null - null], for product: no product, for type: Compou
nd']), ApiValidationError(message='Product is not defined', source=['Tariff: [nul
l - null], for product: no product, for type: Compound']), ApiValidationError(mes
sage='extension type not found', source=['DC Prague']), ApiValidationError(messag
e='extension type not found', source=['DC Berlin'])), validation_status='HAS_ERRO
RS', validation_warnings=None)
```

[SYSTEM] Simulation finished! Captured Result ID: 203631

4. Exporting the rich dashboard page 'Financial and customer performance' (ID: 204648) to Excel...

Exception during Simulation Execution and Export: (500)

Reason:

HTTP response headers: HTTPHeaderDict({'Server': 'nginx', 'Date': 'Sat, 18 Apr 2026 12:06:52 GMT', 'Content-Type': 'application/json', 'Transfer-Encoding': 'chunked', 'Connection': 'keep-alive', 'Vary': 'origin,access-control-request-method,access-control-request-headers,accept-encoding', 'X-Content-Type-Options': 'nosniff', 'X-XSS-Protection': '1; mode=block', 'Cache-Control': 'no-cache, no-store, max-age=0, must-revalidate', 'Pragma': 'no-cache', 'Expires': '0', 'X-Frame-Options': 'DENY', 'Strict-Transport-Security': 'max-age=31536000'})

HTTP response body: {"message": "Cannot invoke \"com.alx.data.experiment.IExperimentRun.getExperimentData()\" because \"run\" is null", "details": {}}

4.b) Modification by Native API variation

```
In [35]: # This path explores setting up an internal ALX variation.
# Step 1 is to ask the server what objects (tables) can be varied natively.

print("--- PATH B: NATIVE API VARIATION METHOD ---")

try:
    print("Querying the server for available variable parameter object types...")

    # We query the server using the cloned scenario ID
    variable_types = api_instance.get_variable_parameter_object_types(scenario_id)

    print("The response of OpenApiApi->get_variable_parameter_object_types:\n")

    # Printing the available tables that can be modified natively
    if variable_types and variable_types.options:
        for option in variable_types.options:
            print(f" - Table Type: {option.name} (ID: {option.id})")
    else:
        print("No variable types found for this scenario.")

except Exception as e:
    print("Exception when calling OpenApiApi->get_variable_parameter_object_type")
```

--- PATH B: NATIVE API VARIATION METHOD ---

Querying the server for available variable parameter object types...

The response of OpenApiApi->get_variable_parameter_object_types:

Exception when calling OpenApiApi->get_variable_parameter_object_types: 'ApiOptionsList' object has no attribute 'options'

=====

DEVELOPMENT TESTS - IGNORE THEM

◀ ————— ▶

```
In [ ]: #Avoid self-signed warnings
import os
import urllib3
```

For self-signed certificates it is necessary to download them and add to the certification path

```
In [ ]: import ssl
import socket

def download_self_signed_cert_no_validation(hostname, port, cert_file):
    context = ssl._create_unverified_context()
    conn = context.wrap_socket(socket.socket(socket.AF_INET), server_hostname=hostname)
    conn.connect((hostname, port))
    der_cert = conn.getpeercert(binary_form=True)
    pem_cert = ssl.DER_cert_to_PEM_cert(der_cert)

    with open(cert_file, 'w') as cert_file:
        cert_file.write(pem_cert)

    print(f"Certificate saved to {cert_file.name}")

# Example usage
download_self_signed_cert_no_validation(SERVER_IP, 443, MY_CERT_FILENAME)
```

```
In [ ]: # !/usr/bin/openssl s_client -showcerts -connect {SERVER_IP}:443 </dev/null 2>/dev/null
```

```
In [ ]: MY_CERT_FILE=os.path.abspath(MY_CERT_FILENAME)
if os.path.isfile(MY_CERT_FILE):
    print(f"Cert file OK: {MY_CERT_FILE} ")
else:
    print(f"Cert file ERROR: {MY_CERT_FILE} ")
MY_CERT_FILE
```

```
In [ ]: os.environ['REQUESTS_CA_BUNDLE'] = MY_CERT_FILE

# Create a PoolManager with a custom CA bundle
http = urllib3.PoolManager(
    cert_reqs='CERT_NONE',      # Enforce cert verification
    ca_certs=MY_CERT_FILE       # Path to your self-signed cert
)
```

```
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)  
http.request("GET", TEST_URL)
```

```
In [ ]: from urllib3 import HTTPConnectionPool
```

```
pool = HTTPConnectionPool(  
    SERVER_IP,  
    port=443,  
    cert_reqs='CERT_REQUIRED',  
    ca_certs=MY_CERT_FILE  
)
```

```
In [ ]: import requests
```

```
# Any request you make now will use your certificate  
response = requests.get(TEST_URL, verify=False)  
print(response.text)
```

```
In [ ]:
```